TECH DEBT

ELEVATED

VELOCITY COLLAPSE

The team is barely shipping. Every new feature requires touching three deeply intertwined legacy modules. Engineers spend more time deciphering old code than writing new features. Estimates keep doubling and the backlog is growing faster than delivery.

Technical debt compounds exactly like financial debt — and the 'interest' is paid as reduced delivery velocity on every future sprint. Martin Fowler's Technical Debt Quadrant distinguishes between deliberate debt (a conscious trade-off) and inadvertent debt (the result of poor practices or lack of knowledge). The DevOps response is to make debt visible: use static analysis tools (SonarQube, CodeClimate) to track code complexity, duplication, and coverage trends over time; add tech debt items to the product backlog with business justification; and allocate a fixed percentage of sprint capacity (typically 20%) to debt reduction. Teams that practise continuous refactoring — treating code quality as a non-negotiable constraint rather than an optional extra — avoid the velocity cliff entirely.

Technical debt is a hidden tax on every future sprint. Ignore it long enough and you stop shipping entirely.

TEAM DISCUSSION

Do you track technical debt formally? Is refactoring scheduled deliberately, or does it keep getting pushed to next sprint?